CS6008: Network Security
Assignment 2: DNS Security — Report

Akshit Kumar (23b0945) Puspendra Uikey (23b1023)

October 24, 2026

Terminal output and annotated packet captures for the verification and attack steps of every phase are shown live in the demo. This report focuses on the design choices and the answers to each phase's report questions.

Setup common to all phases

We use three Docker bridge subnets: the client subnet `172.16.1.0/24`, the resolver subnet `172.16.2.0/24`, and the authoritative subnet `172.16.3.0/24`. The only links between subnets are the two router containers (edge-router and core-router), which are the only nodes that forward traffic. Every other node drops its default route and keeps only the static routes it needs through its router, so a packet cannot jump between subnets except through a router we control. All nameservers and the resolver run BIND 9.20, and every attack program is written in C with libpcap (both capture and `pcap_inject`); we do not use Python or Scapy for any attack.

1 Phase 1 — Building the DNS Hierarchy

Design choices

root-ns is authoritative for the root zone and delegates `com` to tld-ns. tld-ns is authoritative for `com` and delegates `example.com` to domain-ns. domain-ns serves the A records for `example.com` and `www.example.com`, both pointing at the webserver `172.16.1.20`. The resolver is recursive with DNSSEC validation off, and its root hints point at our own root-ns instead of the real root servers. Only the two routers have IP forwarding on, and there is no NAT, so the source addresses stay real end to end.

Report answers

Resolved address, fetched page, and isolation check. From the client, `dig example.com` returns `172.16.1.20` and `curl http://example.com` returns our real example.com page. When we send a query directly to root-ns with `dig @172.16.3.10 example.com`, it fails with "network unreachable": the client has no route to the authoritative subnet, so it cannot talk to any nameserver directly. This shows the subnets are isolated and the client can only reach the hierarchy through the resolver.

Tracing the Root → TLD → Domain referral. In the capture taken on core-router we see the resolver do an iterative lookup:

1. The resolver asks root-ns (`172.16.3.10`) for `example.com`. root-ns does not know the answer, so it replies with a referral: the NS record for `com` and the glue address of tld-ns

(172.16.3.20).
2. The resolver then asks tld-ns (172.16.3.20) for example.com. tld-ns replies with a referral to domain-ns, giving the NS for example.com and the glue address 172.16.3.30.
3. The resolver asks domain-ns (172.16.3.30), which is authoritative and finally answers with A 172.16.1.20.

Everything on the wire is plain cleartext. For each message we can read the query name (example.com), the 16-bit query ID, the source and destination ports, and in the replies the NS names, glue addresses, and the final A record. Nothing is encrypted and nothing is signed, so anyone on the path can read it all.

How the resolver's query reaches root-ns. The resolver has a static route 172.16.3.0/24 via 172.16.2.3 (core-router). So when it needs to reach root-ns at 172.16.3.10, the packet is sent to core-router on the resolver subnet, and core-router (which has forwarding on) forwards it onto the authoritative subnet to root-ns. The ip route output on the resolver and the capture on core-router together show this: the query leaves the resolver, crosses core-router, and arrives at root-ns with the real resolver source address.

What a node on the resolver's path could do. Because the whole exchange is cleartext and unauthenticated, any node sitting on the path between the resolver and the nameservers can read the query (so it knows the ID, the port, and the name being asked) and can send back a fake reply that looks correct. The resolver has no way to tell a real answer from a fake one, so it would accept and cache whatever arrives first and looks right. This is exactly the weakness we attack in Phase 2.

2 Phase 2 — On-Path Response Forgery

Design choices

We carry over the whole Phase 1 setup unchanged and add the attacker at 172.16.2.100 on the resolver subnet, plus the rogue-web server at 172.16.1.30. The only change on the router is a diversion route on core-router (divert.sh) that sends the resolver's inbound replies to the attacker first. The attacker does not forward in the kernel (ip_forward=0), so the diverted packets stop at our libpcap interceptor, which is the only thing that passes them on. The interceptor has a relay mode (passes everything unchanged) and a forge mode (rewrites only the example.com A record to 172.16.1.30).

Report answers

What the resolver cached and what the fetch returned. In relay mode the resolver caches the real 172.16.1.20 and the client gets the real page. In forge mode the resolver caches 172.16.1.30 for example.com, so dig example.com returns 172.16.1.30 and curl returns the rogue page. At the same time www.example.com still resolves to 172.16.1.20, showing only the one target name is poisoned.

Why the attack succeeds. Plain DNS has no authentication: the resolver accepts any reply that has the right query ID, the right port, and matches the question it asked. Normally the hard part for an attacker is guessing the ID and the source port. But because our attacker is on-path, it actually sees the genuine reply go by, so it already knows the ID, the port, and the question. It does not have to guess anything. It just takes the real packet, changes the address inside, and sends it on, and the resolver accepts it as genuine. The on-path position is what removes the guessing that a blind (off-path) attacker would have to do.

How the relay leaves non-target names untouched. In forge mode the interceptor only

changes a packet when the reply's source port is 53 and the question is exactly `example.com` with type A. For every other reply (other names, other types, the referral traffic) it just fixes the checksums and re-injects the packet exactly as it was. So only `example.com` is affected and all other lookups, like `www.example.com`, work normally.

3 Phase 3 — DNSSEC Chain of Trust

Design choices

We sign every zone offline (`sign.sh`/`signer.sh`) using `ECDSAP256SHA256`. Each zone gets a KSK and a ZSK. We sign child before parent so that each child's DS record exists before the parent is signed: we put `example.com`'s DS into the `com` zone and `com`'s DS into the root zone, which gives one unbroken chain from the root down to `example.com`. On the resolver we turn DNSSEC validation on and install our root KSK as the single trust anchor (static-key), and we keep the root hints pointing at our root-ns. The attacker and its binary are left exactly as in Phase 2.

Report answers

What the resolver does with the forged response, and what it checks. When we run the same Phase 2 forge, the resolver rejects it and returns SERVFAIL, so the client's fetch fails (no rogue page). The reason is that the forge changes the A record but cannot change the RRSIG (signature) that covers it, because the attacker has no private key. The resolver validates the A record's RRSIG up the DS/DNSKEY chain to the root trust anchor, finds that the signature does not match the (changed) data, marks the answer bogus, and refuses it. This is different from Phase 2, where the resolver had no way to check the answer and just cached whatever arrived. `www.example.com`, which the attacker does not touch, still validates and resolves normally.

Comparing the wire with Phase 1, and what DNSSEC protects. On the wire the exchange looks the same as Phase 1 plus extra DNSSEC records (RRSIG, DNSKEY, DS). It is still cleartext, so the on-path attacker can still read everything: the query name, the ID, the ports, and the answers. DNSSEC does not hide any of this. What DNSSEC adds is integrity and origin authenticity: the resolver can tell if an answer was changed or did not really come from the owner of the zone. So DNSSEC protects against tampering and forgery, but it does *not* protect confidentiality. Hiding the contents is left for Phase 7.

Signing algorithm and keys in the repo. We chose `ECDSAP256SHA256` because it gives much smaller keys and signatures than RSA for the same security level, which keeps the DNS responses small, and it is modern and widely supported. Its trade-off is that it needs a correct, modern resolver (older software might not support ECDSA) and signing/validation is a bit more CPU heavy per operation than old RSA, though the smaller sizes make up for it. No private key should ever be committed to the repository. Our signing scripts regenerate all keys and signed zones, and `.gitignore` excludes the key files and `.signed` zones. The only key-related thing the resolver needs is the root KSK *public* key, which sits in the trust anchor file.

4 Phase 4 — Signature Stripping and Downgrade

Design choices

We reuse the Phase 3 signed hierarchy and validating resolver. We add a `strip` mode to the interceptor: for `example.com` it rebuilds the reply with every DNSSEC record removed (RRSIG, DNSKEY, DS, NSEC, NSEC3) and the A rewritten to rogue-web, so the answer looks like it

comes from an unsigned zone. We run two deployments: Run A is the normal secure delegation (the `com` zone publishes `example.com`'s DS), and Run B is signed with a `nods` option so `com` does *not* publish `example.com`'s DS.

Report answers

Run A: stripping the signatures (and the DS). In Run A the attack fails and the resolver returns SERVFAIL. Even though we strip all the DNSSEC records off `example.com` on the wire, and even if we also strip the DS out of the `com` referral, the resolver still knows `example.com` is supposed to be signed. It knows this because the DS record for `example.com` lives *inside the signed `com` zone*, and that DS is itself covered by `com`'s signature up to the root. The attacker cannot remove or fake that authenticated "this child is signed" statement without `com`'s key. So when the resolver sees an answer with no signatures for a zone it was told is signed, it treats it as bogus. The attacker cannot make a secure zone look unsigned.

Run B: the one change that reopens the attack. The single change in Run B is that the parent `com` publishes *no* DS record for `example.com` (the zone is still signed, the operator just never gave the DS to the parent). Now there is no authenticated link telling the resolver that `example.com` is signed. The resolver therefore treats `example.com` as an ordinary unsigned (insecure) zone, ignores the signatures that are actually there, and accepts our stripped-and-forged answer. The client gets `172.16.1.30`.

Where downgrade resistance comes from, and the operator mistake. From the two runs it is clear that downgrade resistance does not come from the signatures inside the zone itself; it comes from the DS record in the parent. The DS, sitting in the signed parent zone, is what forces the resolver to demand signatures. The exact operator mistake that throws this away is signing your zone but never publishing its DS in the parent (in the real world, forgetting to give the DS to your registrar). A zone can be perfectly signed and still be hijacked if that one DS is missing.

5 Phase 5 — Zone Walking

Design choices

We reuse the Phase 3 signed, validating deployment. We add five internal hostnames to `example.com` that are not advertised anywhere: `dev`, `vpn`, `mail`, `db`, and `backup`. We sign the zone with plain NSEC (no `-3`), which is the signing choice that makes authenticated denial walkable. The walker is a C/libpcap program (`zonewalk.c`) that queries the authoritative server directly. The fix (`resign-nsec3.sh`) re-signs with NSEC3, reusing the existing keys so the DS already in `com` still matches and the chain of trust stays intact.

Report answers

Recovered names and the exact steps. Starting from only the name `example.com`, the walker follows the NSEC chain. A signed zone proves a name does not exist by returning an NSEC record whose "next name" field points to the next existing name in canonical order. So we query the NSEC record for a name, read the next name out of the reply, then query that name, and repeat until the chain wraps back to the apex. The chain in our zone is:

$$\text{example.com} \rightarrow \text{backup} \rightarrow \text{db} \rightarrow \text{dev} \rightarrow \text{mail} \rightarrow \text{ns} \rightarrow \text{vpn} \rightarrow \text{www} \rightarrow \text{(back to apex)}$$

This recovers all the owner names, including the five hidden ones (`dev`, `vpn`, `mail`, `db`, `backup`), using only DNS queries (no zone transfer, no wordlist) and in about (number of names +2) queries.

Which configuration made the walk possible, and why signed denial leaks. Plain NSEC is what made the walk possible. The reason signed denial leaks, while plain DNS does not, is timing: a signed zone must be able to prove a name does *not* exist, and that proof has to be signed ahead of time, before anyone knows which name will be asked. A proof made in advance cannot be about the asked-for name, so it is instead about the gap between two real names ("nothing exists between `mail` and `ns`"). That gap, by definition, names the real neighbours. Plain DNS just says "no such name" (NXDOMAIN) with nothing attached, so it leaks nothing.

What the fix costs, and what still leaks. The fix is NSEC3, which hashes the names so the "next name" is a hash instead of the real neighbour; our online walker then gets NSEC3 records it cannot follow and recovers nothing, while non-existent names still get a proper authenticated denial. The cost to the operator is extra work: NSEC3 needs hashing at sign time and makes responses and the zone a bit bigger/more complex, and it adds the risk of misconfigured salt/iteration settings. And it is not a perfect fix: an attacker can still collect the NSEC3 hashes and brute-force them offline to recover the names (especially short or guessable ones). NSEC3 raises the cost of enumeration; it does not make it impossible.

6 Phase 6 — Last-Mile Response Forgery

Design choices

We reuse the Phase 3 signed, validating deployment with the upstream untouched, and move the attacker to `172.16.1.100` on the client subnet. We divert the resolver-to-client replies through the attacker with a route on the edge-router (`divert.sh`); there is no ARP spoofing and the client's own configuration is not changed. We test two clients: Client A is an ordinary stub (`dig`/`curl`) and Client B is a locally-validating client (`delv`) that holds the root key as its own trust anchor.

Report answers

Outcome for each client. Client A is hijacked: it takes the forged reply at face value, resolves `example.com` to `172.16.1.30`, and fetches the rogue page. Client B catches the attack: it reports the answer as bogus and refuses it, so it does not land on the rogue page.

Why DNSSEC does not stop this. DNSSEC validation happens at the resolver, upstream of the client. The resolver did validate the real answer correctly. But the hop from the resolver to the client carries no proof of that validation, and this is where the attacker now sits. So the problem is a mismatch: validation happens at the resolver, but the client places its trust in whatever packet appears to come from the resolver. The stub trusts the resolver, and the attacker is impersonating that last hop. Validation happening in one place does not help if trust is placed in a different, unprotected place.

The `ad` flag and what Client B does differently. What tells a stub its answer was validated is the `ad` (authenticated-data) flag in the DNS reply header. The problem is that this is just a single bit in a cleartext, unauthenticated reply on the last mile. Our attacker edits the resolver's already-validated reply in place, so the `ad` bit stays set while the address is a lie; an on-path attacker can leave it set (or set it itself) at will. So a stub that trusts the `ad` bit is trusting something anyone on the path can fake. Client B does not trust the `ad` bit or the resolver at all: it holds the root key and re-validates the signatures itself. Because our forge changes the A record but leaves its RRSIG, Client B's own check of the signature fails and it rejects the answer. (The DNSKEY/DS replies are not forged, so Client B still builds the chain correctly and only the final signature check fails.)

7 Phase 7 — Encrypting the Last Mile (DoT / DoH)

Design choices

We reuse the Phase 6 deployment and add a private CA (`make-certs.sh`). The CA is installed in the client's trust store, so we never use `curl -k` or any other insecure bypass. The resolver is reached by the name `resolver.cs6008.local` (added on the client through `extra_hosts`), and its certificate has exactly that name as its DNS SAN with no IP SAN, so the client can only validate it when connecting by that name. The resolver serves both DoT (TCP 853) and DoH (TCP 443, path `/dns-query`), and we drive the client over DoH. We test two client behaviours: a *strict* client that only speaks DoH and never falls back to cleartext, and a *permissive* client that falls back to cleartext if the encrypted transport is unavailable (but treats a bad certificate as a hard failure).

Report answers

Verification. The client resolves `example.com` over DoH with the certificate validating cleanly against our CA, upstream DNSSEC still holds, and a capture on edge-router shows no cleartext DNS for the lookup (all client-to-resolver traffic is inside TLS on port 443).

Each attempt against each client. We mount three attacks, each against both clients (six combinations):

- **(i) Alter the answer in transit.** Fails for both clients. The answer is inside TLS, so the on-path attacker can relay the encrypted packets but cannot read or change the DNS answer. Both clients get the real page.
- **(ii) Impersonate the resolver with a rogue certificate.** Fails for both clients. The rogue certificate has the right name but is self-signed and not issued by our CA, so certificate validation fails before any answer is exchanged. The strict client refuses; the permissive client also refuses because a bad certificate is a hard failure, not a reason to fall back.
- **(iii) Deny the encrypted transport (block 443/853).** This is the only one that works, and only against the permissive client. With the encrypted ports blocked, the strict client simply fails closed (no answer). The permissive client falls back to cleartext DNS, which the attacker is forging on the last mile, so it is hijacked to `172.16.1.30`.

So out of six combinations, exactly one lands: attempt (iii) against the permissive client. Encryption and authentication stop tampering and impersonation; the only remaining win is a downgrade, and it only works if the client is willing to speak cleartext.

DoT or DoH, and its drawbacks. We deployed both but drive the client over DoH (port 443). DoH's advantage is that it looks like normal HTTPS traffic, so a network cannot easily single it out and block it without blocking all web traffic. Its drawback compared to DoT is that it mixes DNS into web traffic, which makes it harder for a network operator to see and manage DNS, and it carries the extra weight of HTTP on top of TLS. DoT (port 853) is cleaner and easier to reason about because it is a dedicated port, but that same dedicated port is exactly what makes DoT easy to spot and block (as our attempt (iii) shows).

What an encrypted, authenticated last mile still does not protect. It does not protect availability: the transport can be blocked, which either forces a cleartext fallback or denies service, as attempt (iii) shows. It does not hide metadata/traffic analysis: an observer still sees that the client talked to the resolver, when, and roughly how much, even if not the exact names. And it only protects the client-to-resolver hop: everything upstream (resolver to the authoritative servers) is still plain DNS, protected only by DNSSEC, not by this encryption.